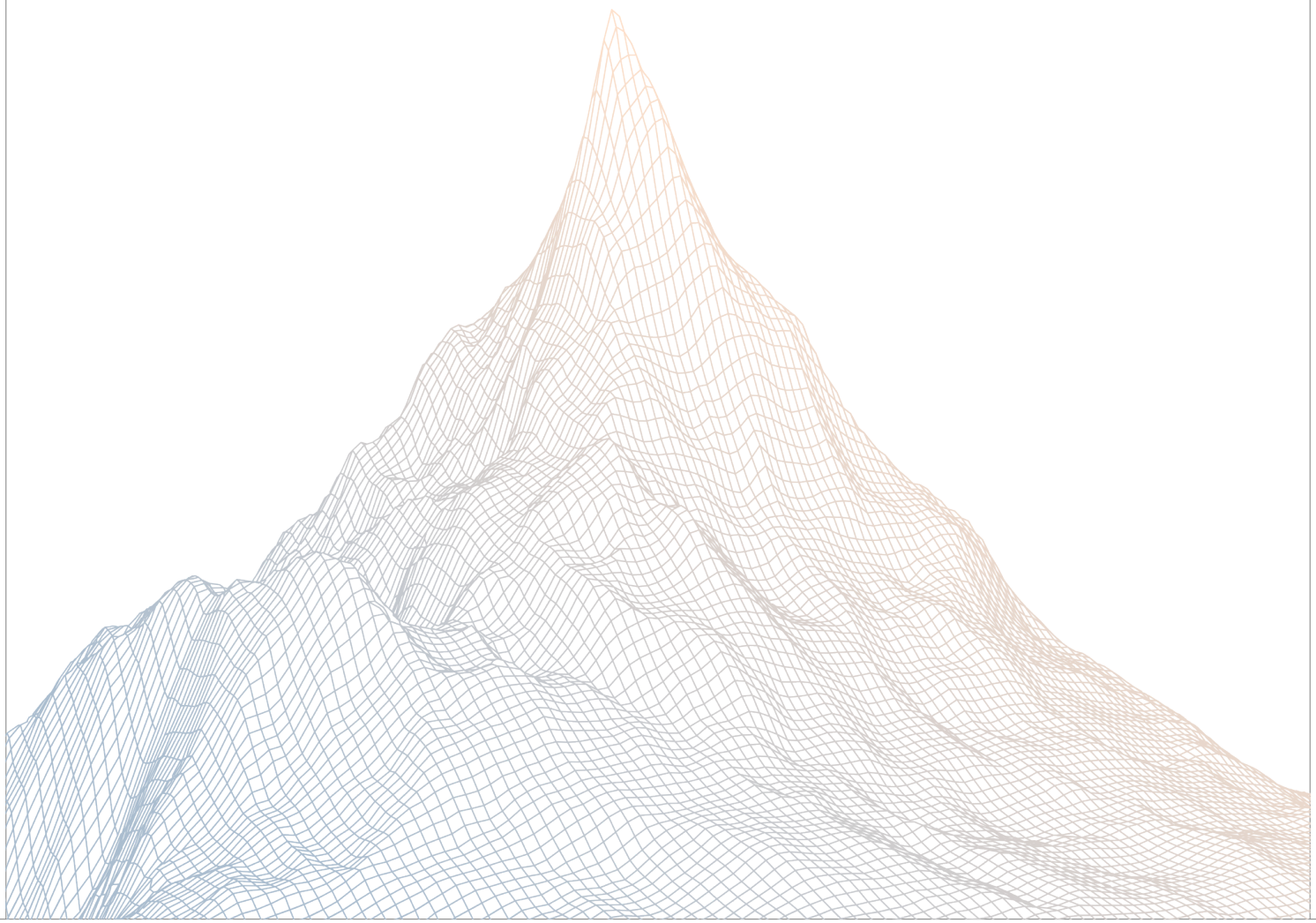
Zenith

Kite AI

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES: August 10th to August 11th, 2026

AUDITED BY: ast3ros
said

Contents

1 Introduction 3

1.1 About Zenith 3

1.2 Disclaimer 3

1.3 Risk Classification 3

2 Executive Summary 4

2.1 About Kite AI 4

2.2 Scope 4

2.3 Audit Timeline 5

2.4 Issues Found 5

3 Findings Summary 6

4 Findings 7

4.1 Informational 7

1 Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at https://zenith.security.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Kite AI

Kite enables AI agents to autonomously transact at scale with cryptographic safety and native x402 compatibility—solving the infrastructure crisis imprisoning the agent economy today.

2.2 Scope

The engagement involved a review of the following targets:

Target	contracts-external
Repository	https://github.com/gokite-ai/contracts-external
Commit Hash	`c4757a0de439df8548238d3fe9329bbe0a0a3107`
Files	`Kite.sol`

2.3 Audit Timeline

August 10, 2026	Audit start
August 11, 2026	Audit end
August 13, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	0
Informational	6
Total Issues	**6**

3

Findings Summary

ID	Description	Status
I-1	LayerZero delegate can diverge from the owner	Acknowledged
I-2	Blacklist does not cover upstream callers	Acknowledged
I-3	Native chain supply invariant is not enforced on-chain	Acknowledged
I-4	Inbound bridge credit to a paused or blacklisted destination burns the source tokens but leaves the destination mint stuck	Acknowledged
I-5	Nothing on-chain enforces that exactly one chain is the native minter	Acknowledged
I-6	Pause roles granted by a previous owner survive an ownership transfer	Acknowledged

4

Findings

4.1 Informational

A total of 6 informational findings were identified.

[I-1] LayerZero delegate can diverge from the owner

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- Kite.sol#L111

Description:

The documented invariant states: `the LayerZero endpoint delegate always equals the new owner.` However, LayerZero `OAppCore` exposes an inherited owner only function:

```
function setDelegate(address _delegate) public onlyOwner {
    endpoint.setDelegate(_delegate);
 }
```

Therefore, the owner can later set the endpoint delegate to a different address.

Recommendations:

Update the invariant to state that the delegate equals the new owner immediately after deployment and after `acceptOwnership`. If permanent equality is intended, override or disable `setDelegate` so the owner cannot assign a different delegate.

Kite AI: Acknowledged, and agreed. Permanent equality between the token owner and the LayerZero delegate is not intended. The contract ensures that the delegate is initially set to the owner and is synchronized to the new owner when `acceptOwnership()` is called, preventing the previous owner from remaining as the delegate after an ownership transfer. Afterward, the owner may intentionally assign a different LayerZero delegate through the inherited `setDelegate()` function. This preserves the ability to delegate LayerZero configuration responsibilities to a separate operational address.

[I-2] Blacklist does not cover upstream callers

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- Kite.sol#L163

Description:

The spec states that a blacklisted address cannot initiate a transfer:

a blacklisted operator cannot move third-party funds

However, `_update` only checks whether `msg.sender` is on the blacklist list. This means a blacklisted address can still call an unblacklisted intermediary contract that is authorized to execute `transferFrom`. In that case, the blacklisted address can be `msg.origin` and the intermediary is `_msgSender()`, so the check passes.

```
    function _update(
        address from,
        address to,
        uint256 value
    ) internal override(ERC20, ERC20Pausable) {
>>>     _requireNotBlacklisted(_msgSender());
        if (from ≠ address(0)) _requireNotBlacklisted(from);
        if (to ≠ address(0)) _requireNotBlacklisted(to);

        super._update(from, to, value);
    }
```

Therefore, blacklisting an address prevents it from interacting with KITE directly, but does not prevent it from causing token movements through other authorized addresses.

Recommendations:

If the intended policy is that a blacklisted address must not be able to initiate or control transfers indirectly, the current check does not enforce that. Otherwise, the specification should clarify that the blacklist applies only to the immediate ERC-20 caller.

Kite AI: Acknowledged. The intended policy applies to the immediate ERC-20 caller, sender, and recipient, rather than every upstream address involved in a transaction.

A blacklisted holder still cannot transfer its own tokens because from is checked, cannot receive tokens because to is checked, and cannot interact with KITE directly because `_msgSender()` is checked.

[I-3] Native chain supply invariant is not enforced on-chain

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- Kite.sol#L43-L55

Description:

The intended invariant is that KITE's initial supply is minted only on the native-chain deployment. However, whether a deployment is considered native is determined entirely by the deployer-provided `_isNativeChain` flag:

```
    constructor(
        address _lzEndpoint,
        address _owner,
        bool _isNativeChain,
        uint256 _initialSupply
    ) OFT("Kite", "KITE", _lzEndpoint, _owner) Ownable(_owner) {
>>>     isNativeChain = _isNativeChain;
        if (_isNativeChain) {
            _mint(_owner, _initialSupply);
        } else if (_initialSupply != 0) {
            revert InvalidInitialSupply(_initialSupply);
        }
    }
```

There is no validation against `block.chainid`. Consequently, any deployment can set `_isNativeChain = true` and mint an arbitrary initial supply, so global supply conservation depends on correct deployment configuration rather than being enforced by the contract.

Recommendations:

Bind the native chain designation to the expected chain ID.

Kite AI: Acknowledged. We do not plan to bind the native-chain designation to `block.chainid`.

`isNativeChain` is immutable and only affects construction; the contract has no post-deployment mint function. Official deployments are created through a controlled deployment process, and the chain, constructor arguments, initial supply, and deployed address are verified before any LayerZero peers are configured or the token is activated.

A misconfigured or third-party deployment cannot join the official KITE deployment mesh. We will retain this as an explicit deployment and verification requirement.

[I-4] Inbound bridge credit to a paused or blacklisted destination burns the source tokens but leaves the destination mint stuck

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- Kite.sol#L158-L168

Description:

Bridging burns on the source chain immediately (`_debit`) and mints on the destination later (`_credit`). If the destination is paused or the recipient is blacklisted at delivery time, the mint reverts while the source burn has already happened, so the tokens exist on neither chain for that window. Recovery is owner-gated (unpause / un-blacklist) and manual.

Recommendations:

Document the recovery path (re-run `lzReceive` after the condition clears) for operators.

Kite AI: Acknowledged, and agreed. This is the intended behavior of the control: a paused contract or blacklisted recipient must not receive an inbound credit. The contract reverts in `_credit` before the message is consumed or any partial credit occurs, leaving the transfer pending rather than lost. Once the pause or blacklist is cleared, the message can be re-executed and credited in full. This flow is covered by `testBlacklistedRecipientCannotBeCreditedOnDestination`. We accept the recommendation and will add the recovery procedure—clear the applicable restriction, then re-execute the pending message—to the operator runbook.

[I-5] Nothing on-chain enforces that exactly one chain is the native minter

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- Kite.sol#L43-L55

Description:

`isNativeChain` and `_initialSupply` are plain constructor inputs. Any deployment with `_isNativeChain = true` mints `_initialSupply`, and nothing on-chain stops two chains from being deployed as native, which would put twice the intended supply into circulation once bridging links them. This issue becomes live the moment remote deployments are added, which the stated invariants ("any remote-chain deployment always starts at zero supply") contemplate.

Recommendations:

If remote deployments are ever added, assert `totalSupply() == 0` on every non-native chain right after deploy and before wiring.

Kite AI: Acknowledged, agreed on the residual, and the enforceable half is already enforced: a non-native deployment cannot mint — _isNativeChain = false with any nonzero _initialSupply reverts with InvalidInitialSupply (Kite.sol#L52-L54), so every remote deployment provably starts at zero supply. What no contract can enforce is whether another chain was also deployed as native: there is no on-chain view of a remote deployment at construction time, so "exactly one native chain" is inherently a deployment-procedure invariant. It is also gated twice — a second native deployment only reaches circulation if the owner separately wires peers to it, which is a deliberate owner action, not a consequence of deploying. We accept your recommendation as the procedural control: totalSupply() == 0 verification on every non-native deployment before wiring is in our deployment checklist.

[I-6] Pause roles granted by a previous owner survive an ownership transfer

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- Kite.sol#L109-L112
- Kite.sol#L64-L69

Description:

`acceptOwnership` re-points the LayerZero delegate at the new owner so the old owner keeps no endpoint authority after the handoff. Pause roles do not get the same treatment. `setPauseRole` grants live in the `pauseRoles` mapping, and nothing in `acceptOwnership` clears them, so every pause role the old owner handed out stays active after ownership changes hands. There is also no on-chain list of pause-role holders, a new owner can only reconstruct the set from `PauseRoleUpdated` event history. The behavior is documented in the NatSpec and in `DENYLIST_TESTING.md` but the residual-authority guarantee is inconsistent between the delegate and the pause roles.

Recommendations:

- Mirror the delegate handling: clear the pause-role set in `acceptOwnership` so the new owner starts clean, or at least make the role set enumerable so it can be audited on-chain.
- If kept as-is, document the mandatory post-handoff review.

Kite AI: Acknowledged, but deliberate. The delegate is a single implicit slot whose only correct value is the current owner; pause roles are configured guardians that exist to brake faster than a multisig can sign, clearing them on handoff would leave the token unguarded until each role is re-granted. The residual risk is bounded: a pause role can only pause, so the worst case is a DoS the owner reverses with one `unpause()`, and `PauseRoleUpdated` makes the live set reconstructible.